

Função – Escopo e tempo de vida de variáveis

Dois conceitos importantes que envolvem variáveis declaradas em funções e programas são escopo e tempo de vida. O primeiro diz respeito à abrangência da variável, ou seja, em quais partes do programa a variável pode ser acessada e manipulada.

Escopo pode ser informalmente o local onde no programa uma variável é conhecida. Por exemplo, se você tenta usar uma variável que foi definida em um escopo local fora desse escopo vai obter um `NameError` e o seu programa será interrompido.

Função – escopo local

Quando criamos uma variável dentro da definição de uma função, a variável tem escopo local, isto significa que somente o código dentro daquela função reconhece o nome e pode usar os valores a ele atribuídos.

Os parâmetros são os nomes que recebem os valores, argumentos, usados na chamada de uma função, são declarados no cabeçalho da definição da função, e também são nomes do escopo local da função.

No exemplo abaixo, a variável `vlr_desconto` somente é reconhecida dentro da função `calcular_desconto()`.

```
def calcular_desconto(valor,desconto):  
    vlr_desconto = valor * desconto / 100  
    return vlr_desconto
```

Função – escopo local

Do mesmo modo, a variável `valor_total` é válida apenas dentro da função `totalizar_item()`. A essa característica damos o nome de escopo local.

```
def totalizar_item(vlr_unit,quantidade):  
    valor_total = vlr_unit * quantidade  
    return valor_total
```

Função – escopo global

Há outro tipo de escopo, denominado global, em que a variável é reconhecida em qualquer parte do programa, seja ela uma função ou um trecho do código principal.

Criamos variáveis globais quando atribuímos um valor a um nome fora do corpo das funções do nosso programa (normalmente no começo do programa), também podemos dizer que a variável tem escopo global.

Essas variáveis podem ser usadas ao longo de todo o programa. Se você desejar criar uma nova variável global, ou reatribuir (alterar) o valor de uma variável global de dentro de uma função, em Python, é necessário usar a instrução global.

Função – escopo global

```
x = 0 # x é uma variável global
```

```
def keyPressed():
```

```
    # Queremos alterar o valor da variável global x.
```

```
    global x
```

```
    x = x + 1
```

```
    return x
```

```
print(keyPressed())
```

Função – escopo global e local

valor_unit = 100 # valor_unit é uma variável global

desconto = 10 # desconto é uma variável global

quantidade = 3 # quantidade é uma variável global

def calcular_desconto():

 valor_desconto = valor_unit * desconto / 100 # valor_desconto é uma variável local da função

e não tem qualquer relação com a variável de mesmo nome declarada no corpo principal do programa

 return valor_desconto

def totalizar_item():

 valor_total = valor_venda * quantidade # a variável valor_total da função totalizar_item() não se refere à mesma variável fora dela.

 return valor_total

valor_desconto = calcular_desconto()

valor_venda = valor_unit - valor_desconto

valor_total = totalizar_item()

print('Valor do desconto: %8.2f' % valor_desconto)

print('Valor de venda: %8.2f' % valor_venda)

print('Valor total do item: %10.2f' % valor_total)

Conselhos sobre variáveis globais

É comum escutarmos que devemos usar variáveis globais com cautela.

O uso irresponsável de variáveis globais pode provocar valores inesperados em pontos do programa.

Em grandes projetos e com muitos desenvolvedores, o uso de variáveis globais é evitado, com o argumento de que seu uso viola certas “boas práticas” de engenharia de software.

É considerado desejável o máximo encapsulamento das partes de um programa, sendo elas independentes, com interfaces claras e adequadas para receber e devolver valores.

Conselhos sobre variáveis globais

O uso de variáveis de escopo global não é recomendado por diversos motivos, mas podemos elencar três como principais:

- o primeiro é a possibilidade de alteração indevida do valor armazenado nelas.
- o segundo é a confusão que pode surgir no caso de uso de uma variável local em uma função que possua o mesmo nome de uma variável global, como é o caso do nosso exemplo. Essa confusão se dá pelo fato de, em determinado momento, não sabermos se estamos manipulando uma variável local ou uma variável global.
- E o terceiro motivo é a dependência que as funções possuem em relação ao corpo principal do programa, pois as variáveis globais que elas utilizam precisam ter seus nomes definidos da mesma forma nele.

Inserindo DocStrings

É uma boa prática documentar o código, descrevendo qual o objetivo da função e o que representam os parâmetros.

```
def Pitagoras(a, b, calc='hip') -> float:
    '''
    Esta função tem como objetivo automatizar o cálculo de pitagoras,
    onde tentamos achar o valor de um dos lados de um triângulo retângulo.
    Parametros:
    -----
    * ``a`` = int or float
        Valor conhecido de um dos lados do triângulo retângulo,
        podendo ser um cateto ou a hipotenusa.
    * ``b`` = int or float
        Valor conhecido do segundo lado do triângulo retângulo,
        podendo ser um cateto ou a hipotenusa.
    * ``calc`` = 'hip', 'cat', default='hip'
        Valor que deseja encontrar sendo:
        - hip = hipotenusa
        - cat = cateto
    '''
```

Como visualizar docstrings no Python

help(Pitagoras)

print(Pitagoras. doc)

```
Esta função tem como objetivo automatizar o cálculo de pitagoras,  
onde tentamos achar o valor de um dos lados de um triângulo retângulo.
```

```
Parametros:
```

```
-----
```

```
* ``a`` = int or float
```

```
    Valor conhecido de um dos lados do triângulo retângulo,  
    podendo ser um cateto ou a hipotenusa.
```

```
* ``b`` = int or float
```

```
    Valor conhecido do segundo lado do triângulo retângulo,  
    podendo ser um cateto ou a hipotenusa.
```

```
* ``calc`` = 'hip', 'cat', default='hip'
```

```
    Valor que deseja encontrar sendo:
```

- hip = hipotenusa
- cat = cateto

Passos para criar um pacote

- Crie uma pasta nova no seu projeto.
- Dê um nome para a pasta, por exemplo `minhas_funcoes`.
- Dentro dessa pasta, crie um arquivo vazio chamado `__init__.py`.
- Crie um arquivo Python dentro da pasta, como `matematica.py`.
- Escreva sua função dentro de `matematica.py`

Como usar as funções

- No seu arquivo principal (fora da pasta), importe a função.
- Use o comando `from` indicando o nome da pasta e do arquivo:

```
from minhas_funcoes.matematica import somar  
resultado = somar(2, 3)  
print(resultado)
```

Atividade

Referência

Alves, William P. Programação Python: aprenda de forma rápida. Disponível em: Minha Biblioteca, Grupo GEN, 2021.

([https://integrada.minhabiblioteca.com.br/reader/books/9786558110149/epubcfi/6/20\[%3Bvnd.vst.idref%3Dmiolo6.xhtml\]!/4](https://integrada.minhabiblioteca.com.br/reader/books/9786558110149/epubcfi/6/20[%3Bvnd.vst.idref%3Dmiolo6.xhtml]!/4))